一、视图层

二、教学目标

1、完成编辑功能

2、完成删除功能

3、掌握jsonResponse 案例

4、掌握上传文件

三、编辑

3.1 概述

目前我们已经完成了用户列表功能 编辑还没完成

步骤:

1. userlist.html中写好前往编辑路由
2. 编写urls.py 路径的匹配
3. 在views.py中获取前端传递过来的id
4. 根据id查询对象信息
5. 完成修改提交

3.2 补充代码

1. `templates` **文件夹中** `userlist.html`

```
<a href="{% url 'edit' userli.id %}">编辑</a>
```

2. `urls.py`

```
re_path('edit/(\d+)',views.edit,name='edit'),
```

3. `views.py`

```python
def edit(request,edit_id):
    #根据id查询对象
    res = models.User.objects.filter(id=edit_id)
    if res:
        #查询第一个值
        user_obj = res[0]
        #返回到edit.html 带对象
        return render(request,'edit.html',locals())
    else:
        return HttpResponse('暂无数据')
```

4. `templates`文件夹中创建`edit.html` -> 回显数据

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static '/bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "/js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "/bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<h1 class="text-center">编辑用户</h1>
<div class="row">

    <div class="col-md-8 col-md-offset-2">

        <form action="" method="post">
            <p>用户名:<input type="text" value="{{ user_obj.username }}"
name="username" class="form-control"></p>
            <p>密码:<input type="text" value="{{ user_obj.password }}"
name="password" class="form-control"></p>


            <input type="submit" class="btn btn-primary btn-block">

        </form>

    </div>
</div>
</body>
</html>
```

完成测试

5. **修改提交**

发送请求的时候也是当前的urls的方法 所以可以在原来的`views.py`添加一个判断即可 -> POST请求

```
def edit(request,edit_id):

    #根据id查询对象  查询所有查询数据的集合 一般根据id查询只会返回一条
    res = models.User.objects.filter(id=edit_id)
    if res:
        #查询第一个值
        user_obj = res[0]

        if request.method == 'POST':
            username = request.POST.get('username')
            password = request.POST.get('password')
            # 相当于发送update 请求 这里可以不用根据id查询 因为上面已经做了判断
            res.update(username=username, password=password)
            #编辑成功后 重定向到用户列表页面
            return redirect('/userlist/')
        #返回到edit.html 带对象
        return render(request,'edit.html',locals())

    else:
        return HttpResponse('暂无数据')
```

完成测试

四、删除

4.1 概述

目前我们已经完成了用户列表功能 删除还没完成

步骤:

```
1. userlist.html中写好前往删除路由
2. 编写urls.py 路径的匹配
3. 在views.py中获取前端传递过来的id
4. 根据id查询对象信息
5. 完成删除
```

4.2 补充代码

1. `templates` **文件夹中** `userlist.html`

```
<!-- 这里注意哈 我没用反向解析 直接使用url路由全名称 为了区分我们不用反向解析怎么实现  -->
<a href="/delete/{{ userli.id }}/">删除</a></td>
```

2. `urls.py`

```
#删除 这里就不用别名 但是我们用正则有名分组 目的就是让大家知道怎么应用和传参
re_path('delete/(?P<delete_id>\d+)/',views.delete),
```

3. `views.py`

```
#删除
#有名正则有名分组 传参必须要用指定的名字delete_id
def delete(request,delete_id):
    #根据id查询对象调用删除函数  _ 返回受影响的行数
    res, _ = models.User.objects.filter(id=delete_id).delete()
    print(res)
    if res>0:
        return redirect('/userlist/')
    else:
        # 处理未删除任何记录的情况
        return HttpResponse("删除失败，记录不存在")
```

完成测试

用户信息

id	用户名	密码	操作
1	daxi	12345	编辑 删除
2	admin	123	编辑 删除

五、路由分发

> 前言:
>
> 目前我们发现我们的代码都是写在一起的,而且是很乱
>
> 假如真实项目是这样开发的话 就会导致代码都混在一起
>
> 正常来说 在每个应用当中都应该有自己的路由 自己的模版 自己的视图 .. 等等
>
> 所以djaong能做到很好的分组开发(也就是每个人只写自己的app)
>
> 解决方案:
>
> 利用路由分发, 根据路由就不在做路由匹配(路由和视图函数的对应关系),而是做分发处理
>
> 识别url是来自于哪个应用,就直接交给对应的子路由

5.1 3.1 概述

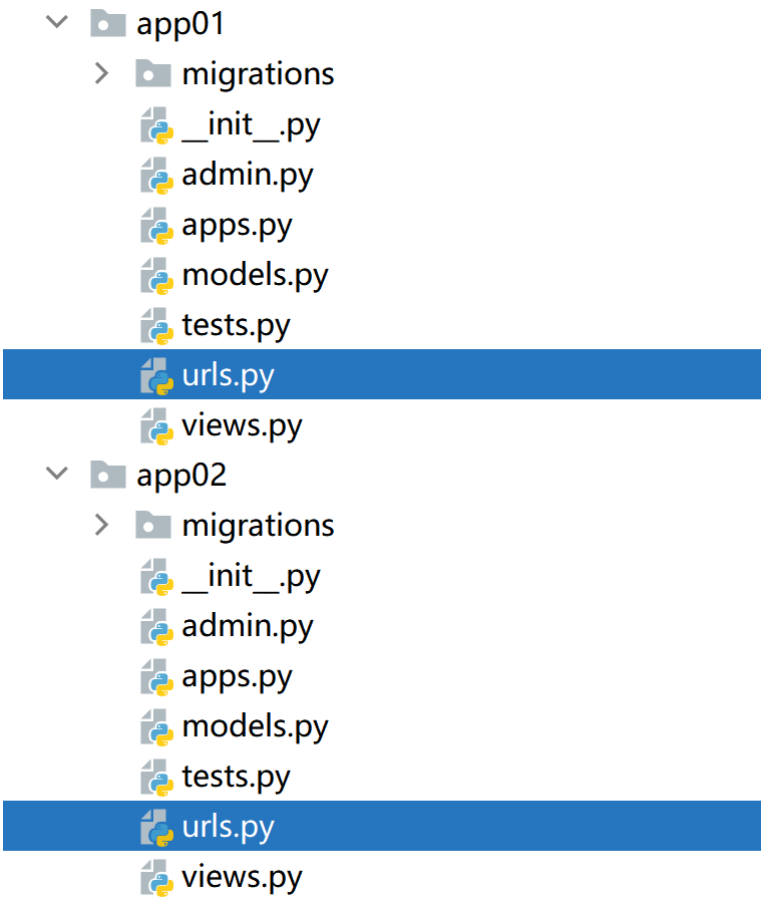
路由分发是 Django 中将不同 URL 请求分配到对应应用模块的机制，通过 `include()` 函数实现模块化管理。核心作用是 **解耦项目与应用的路由配置**，提升代码可维护性。

5.2 3.2 创建应用

1. **创建app02应用**

```
python manage.py startapp app02
```

2. **在每个应用中都创建 urls.py**



3. `在app01的urls.py` **完善代码**

```
from django.contrib import admin
from django.urls import path,re_path
from . import views

urlpatterns = [

    path("python/",views.python)
]
```

4. `在app02的urls.py` **完善代码**

```
from django.contrib import admin
from django.urls import path, re_path
from . import views

urlpatterns = [

    path("python/", views.python)
]
```

5. **在总路由配置**

```
from django.contrib import admin
from django.urls import path,re_path,include
from . import views
#将应用中的urls导入过来 并且起别名
from app01 import urls as app01_urls
from app02 import urls as app02_urls

urlpatterns = [
    #演示路由分发
    path('app01/',include(app01_urls)),
    path('app02/',include(app02_urls)),
]
```

6. **在地址栏中输入地址进行测试**

```
http://127.0.0.1:8001/app01/python/
http://127.0.0.1:8001/app02/python/
```

7. **优化**

- 可以使用名称空间
- 可以不用使用导入应用的urls
- 使用　应用名.urls

```
from django.contrib import admin
from django.urls import path,re_path,include
from . import views
#将应用中的urls导入过来 并且起别名
#from app01 import urls as app01_urls
#from app02 import urls as app02_urls

urlpatterns = [
    #演示路由分发
    path('app01/',include('app01.urls')),
    path('app02/',include('app02.urls')),
]
```

注意: 当多个应用出现了相同的别名的时候做反向解析时候不能识别到前缀是app01还是app02 这个时候就可以使用名称空间

一般情况下如果有多个应用,我们起别名时最好在前面加上应用名做区别,这样的话就不会冲突,也不需要使用名称空间

这样的话最好加上`namespace=`参数

例如:

```
path('app01/',include('app01.urls',namespace='app01')),
path('app02/',include('app02.urls',namespace='app02')),
```

- **5.2.0.1 优势：**
    - **模块化**：各应用独立维护路由，避免代码臃肿。
    - **可维护性**：修改应用路由时无需改动项目总路由。
    - **团队协作**：不同开发者可并行处理不同应用的路由
- **5.2.0.1 总结**

  路由分发通过 `include()` 将 URL 请求分层处理，适合多应用项目开发，保持代码清晰。

六、视图层

> 视图层是整个django项目的核心,所有后端的代码都在这里实现
>
> 一个视图函数,简称"视图" 是一个简单python函数 接受web请求,进行处理,返回web响应
>
> 视图的第一个参数必须为HttpRequest实例(request对象),并且返回的响应必须为HttpResponse对象或者子对象

- HttpResponse 返回字符串类型
- render 返回html页面,并且在返回之前给html传值
- redirect 重定向
- **jsonResponse** 返回JSON

6.1 新建一个项目`django_views`

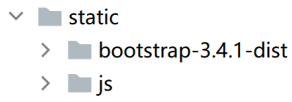
这里可以创建虚拟环境 可以不创建 这个自行决定

6.2 创建`static`文件夹

- 注册

```
TEMPLATES = [
    {
        .....
            'builtins':[
                'django.templatetags.static'
            ]

        },
    },
]
```

- 放入我们之前的bootstarp的文件夹



6.3 创建`templates`文件夹

- 注册

```
TEMPLATES = [
    {
       ......
        'DIRS': [BASE_DIR / 'templates'],
        ....

    },
]
```

6.4 创建app01 应用

```
python manage.py startapp app01
```

6.5 查看源码分析

- **HttpResponse**

```
class HttpResponse(HttpResponseBase):
    """
    An HTTP response class with a string as content.

    This content can be read, appended to, or replaced.
    """

    streaming = False

    def __init__(self, content=b'', *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Content is a bytestring. See the `content` property methods.
        self.content = content

    def __repr__(self):
```

```python
def __init__(self, content_type=None, status=None, reason=None, charset=None, headers=None):
    self.headers = ResponseHeaders(headers or {})
    self._charset = charset
    if content_type and 'Content-Type' in self.headers:
        raise ValueError(
            "'headers' must not contain 'Content-Type' when the "
            "'content_type' parameter is provided."
        )
    if 'Content-Type' not in self.headers:
        if content_type is None:
            content_type = 'text/html; charset=%s' % self.charset
        self.headers['Content-Type'] = content_type
    self._resource_closers = []
    # This parameter is set by the handler. It's necessary to preserve the
    # historical behavior of request_finished.
    self._handler_class = None
    self.cookies = SimpleCookie()
    self.closed = False
    if status is not None:
        try:
            self.status_code = int(status)
        except (ValueError, TypeError):
```

esponseBase › init ()

发现我们的源码就是帮助我们添加了请求头等等之类的信息

- **render**

```python
def render(request, template_name, context=None, content_type=None, status=None, using=None):
    """
    Return a HttpResponse whose content is filled with the result of calling
    django.template.loader.render_to_string() with the passed arguments.
    """
    content = loader.render_to_string(template_name, context, request, using=using)
    return HttpResponse(content, content_type, status)
```

返回的其实也就是HttpResponse

- **redirect**

```python
def redirect(to, *args, permanent=False, **kwargs):
    """
    Return an HttpResponseRedirect to the appropriate URL for the arguments
    passed.

    The arguments could be:

        * A model: the model's `get_absolute_url()` function will be called.

        * A view name, possibly with arguments: `urls.reverse()` will be used
          to reverse-resolve the name.

        * A URL, which will be used as-is for the redirect location.

    Issues a temporary redirect by default; pass permanent=True to issue a
    permanent redirect.
    """
    redirect_class = HttpResponsePermanentRedirect if permanent else HttpResponseRedirect
    return redirect_class(resolve_url(to, *args, **kwargs))
```

```python
class HttpResponseRedirect(HttpResponseRedirectBase):
    status_code = 302


class HttpResponsePermanentRedirect(HttpResponseRedirectBase):
    status_code = 301


class HttpResponseNotModified(HttpResponse):
    status_code = 304
```

看到状态码是帮我们设置好的 而且也是返回的是HttpResponse

解释：

从 **HttpResponse** 的使用过程可知，如果要生成网页内容，就需要将 HTML 语言以 字符串的形式表示，如果网页内容过大，就会增加视图函数的代码量，同时也没有体现模板的作用，因此Django在此基础上进行了封装处理，定义了函数 **render** 和 **redirect**。

6.6 jsonResponse 案例

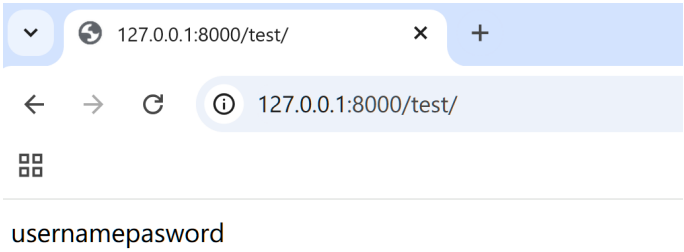
1 在urls.py

```
from django.contrib import admin
from django.urls import path
from app01 import views



urlpatterns = [
    path('admin/', admin.site.urls),
    path('test/',views.test)
]
```

2 在app01应用中views.py

```
def test(request):

    dict1 = {'username': 'admin','pasword':123}
    return HttpResponse(dict1)
```

3 发现问题只得到健没有值?



4 想要完美的展示我们的json数据需要导入json模块

```
import json
def test(request):

    dict1 = {'username': 'admin','pasword':123}
    #通过json模块 将字典转成json 因此前端才可以识别到
    json_str = json.dumps(dict1)
    return HttpResponse(json_str)
```

测试输出

{"username": "admin", "pasword": 123}

5 如果我们数据增加一点然后是一个列表而且是中文看会发生什么事情

```
dict1 = {'username': 'admin','pasword':123,'hobby':['唱','跳']}
```

测试输出

{"username": "admin", "pasword": 123, "hobby": ["\u5531", "\u8df3"]}

解决方案:

ensure_ascii=False

```
json_str = json.dumps(dict1,ensure_ascii=False)
```

6 导入jsonResponse

```
from django.shortcuts import render,HttpResponse,redirect

from django.http import JsonResponse


# Create your views here.
import json

def test(request):

    #dict1 = {'username': 'admin','pasword':123,'hobby':['唱','跳']}
    #通过json模块 将字典转成json 因此前端才可以识别到
    #json_str = json.dumps(dict1,ensure_ascii=False)
    # return JsonResponse(list1,json_dumps_params={'ensure_ascii':False})
    list1 = [1,2,3,4]
    #如果传递的不是字典 就会报错 In order to allow non-dict objects to be serialized
set the safe parameter to False.
    # 解决方案
    return JsonResponse(list1,safe=False)

#通过源码分析
# isinstance (对象,类)
```

6.7 上传文件

- from 表单上传文件数据怎么获取?

前端:

> 表单如果想要上传类型的数据
>
> method 指定为post
>
> enctype: encodetype编码类型的意思, 默认值是: application/x-www-form-urlencoded 只能获取文本
>
> 想要获取文件和文本数据要修改成 **multipart/form-data** 指的就是表单数据由多个部分组成
>
> 既又文本数据,又有文件等二进制数据

1 **templates 创建 file.html**

```
<body>
<form action="" method="post"enctype="multipart/form-data">
    <p>用户名:<input type="text" name="username"></p>
    <p>文件:<input type="file" name="file"></p>
    <input type="submit">
</form>
</body>
```

后端:

> 获取文件数据用request.FILES获取文件数据

2 django_views - > urls.py

```
from django.contrib import admin
from django.urls import path
from app01 import views



urlpatterns = [
    path('admin/', admin.site.urls),
    path('test/',views.test),
    #文件
    path('file/',views.file)
]
```

3 app01.views.py

```
#上传文件
def file(request):

    if request.method =='POST':
        username = request.POST.get('username')
        #这样的方式 只能获取文件名
        # file = request.POST.get('file')

        #获取文件对象
        file = request.FILES.get('file')

        print(username)
        print(file)
        #下载到本地
        with open(file.name,'wb') as f:
            for i in file:
                f.write(i)
    return render(request,'file.html')
```

总结:

request对象的方法

request.POST·获取 post 请求的普通的键值对数据

request.POST.get()·获取列表中最后一个元素的数据.

request.POST.getlist()·获取整个列表request.FILES·获取文件数据

request.GET·获取 get 请求携带的数据

request.path:·获取请求路径,

request.path_info·:获取请求路径,

request.get_full_path()·:获取请求路径,可以获取完整的携带数据的路径,

七、作业

7.1 完成本天的案例